TensorFlow Fundamentals – Complete Notes (Simple and Clear)

TensorFlow is an open-source Python library developed by Google for Machine Learning and Deep Learning. It is used to build, train, and deploy AI models.

The main data structure in TensorFlow is called a **Tensor**.

A tensor is a multi-dimensional array that stores data. It is similar to a NumPy array but has additional advantages:

- It can run on CPUs, GPUs, and TPUs.
- It is optimized for machine learning.
- It supports automatic differentiation for training neural networks.

A tensor can have different dimensions.

A **Scalar** is a single value.

Example:

```
scalar = tf.constant(7)
```

Output:

```
7
```

Rank (Dimensions): 0

A **Vector** is a one-dimensional collection of values.

Example:

```
vector = tf.constant([10, 20, 30])
```

Output:

```
[10 20 30]
```

Rank: 1

A **Matrix** is a two-dimensional collection of rows and columns.

Example:

```
matrix = tf.constant([
   [1,2],
```

```
    [3,4]
])
```

Output:

```
[[1 2]
 [3 4]]
```

Rank: 2

A **Higher-Dimensional Tensor** has three or more dimensions.

Example:

```
tensor = tf.constant([
    [[1,2],[3,4]],
    [[5,6],[7,8]]
])
```

Shape:

(2,2,2)

Rank:

3

A tensor has four important properties.

Shape tells us how many elements are present in each dimension.

Example

```
tensor.shape
```

Output

(2,2)

It means there are 2 rows and 2 columns.

Rank tells us the number of dimensions.

Example

tensor.ndim

Output

2

Size tells us the total number of elements.

Example

tf.size(tensor)

Output

4

Datatype tells us what type of values the tensor stores.

Example

tensor.dtype

Output

int32

TensorFlow provides two ways to create tensors.

The first is **tf.constant()**.

A constant tensor cannot be modified after it is created.

Example

x = tf.constant([10,20,30])

Trying to change its values will result in an error.

The second is **tf.Variable()**.

A variable tensor can be modified after creation.

Example

```
x = tf.Variable([10,20,30])
```

Changing a value

```
x[0].assign(100)
```

Output

```
[100 20 30]
```

Variable tensors are mainly used for neural network weights because their values change during training.

TensorFlow can also generate random values.

Random tensors are mainly used to initialize weights in deep learning models.

Example

```
g = tf.random.Generator.from_seed(42)

random_tensor = g.normal(shape=(2,3))
```

Every time you use the same seed, you get the same random values.

TensorFlow can shuffle data.

Shuffling is important because if data is always in the same order, the model may learn unwanted patterns.

Example

```
data = tf.constant([
    [1,2],
    [3,4],
    [5,6]
])

tf.random.shuffle(data)
```

TensorFlow can quickly create tensors filled with ones.

Example

```
tf.ones((2,3))
```

Output

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

It can also create tensors filled with zeros.

Example

```
tf.zeros((2,3))
```

Output

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

TensorFlow can convert NumPy arrays into tensors.

Example

```
import numpy as np

arr = np.array([1,2,3,4])

tensor = tf.constant(arr)
```

Now the NumPy array becomes a TensorFlow tensor.

TensorFlow supports indexing just like Python lists.

Example

```
x = tf.constant([
    [10,20],
    [30,40]
])
```

First row

```
x[0]
```

Output

```
[10 20]
```

Second column

x[:,1]

Output

[20 40]

Sometimes we need to add an extra dimension.

Example

```
x = tf.constant([
    [1,2],
    [3,4]
])
```

Shape

(2,2)

Adding a new dimension

tf.expand_dims(x, axis=-1)

New Shape

(2,2,1)

We can also use

x[..., tf.newaxis]

Both methods produce the same result.

TensorFlow supports normal mathematical operations.

Addition

x + 10

Subtraction

x - 5

Multiplication

x * 10

Division

x / 2

TensorFlow also provides functions.

Example

tf.multiply(x,10)

It produces the same result as

x * 10

Matrix multiplication is different from normal multiplication.

Normal multiplication multiplies each element individually.

Matrix multiplication follows mathematical rules.

The number of columns in the first matrix must equal the number of rows in the second matrix.

Example

```
A = tf.constant([
    [1,2],
    [3,4]
])

B = tf.constant([
    [5,6],
    [7,8]
])

tf.matmul(A,B)
```

You can also write

A @ B

Both give the same result.

If dimensions do not match, TensorFlow throws an error.

Sometimes we transpose a matrix.

Transpose swaps rows and columns.

Example

Original

1 2
3 4
5 6

Transpose

1 3 5
2 4 6

TensorFlow function

tf.transpose(A)

TensorFlow can also change the datatype of a tensor.

Example

x = tf.constant([1,2,3])

Datatype

int32

Convert to float

tf.cast(x, tf.float32)

Output datatype

float32

This is useful because many deep learning models work with floating-point numbers.

Absolute value removes negative signs.

Example

x = tf.constant([-5,-10,20])

tf.abs(x)

Output

[5 10 20]

Aggregation means reducing many values into one value.

Minimum

tf.reduce_min(x)

Maximum

tf.reduce_max(x)

Mean

tf.reduce_mean(x)

Sum

tf.reduce_sum(x)

These functions are commonly used while analyzing datasets.

To find the position of the maximum value

tf.argmax(x)

Suppose

[2,10,5]

Output

1

because 10 is at index 1.

To find the position of the minimum value

tf.argmin(x)

TensorFlow can remove unnecessary dimensions.

Example

Shape

(1,1,1,50)

Using

tf.squeeze(x)

New Shape

(50,)

This removes all dimensions whose size is 1.

One-Hot Encoding converts categorical values into binary vectors.

Suppose

Cat = 0
Dog = 1
Bird = 2

One-hot encoding

Cat -> [1 0 0]

Dog -> [0 1 0]

Bird -> [0 0 1]

TensorFlow function

tf.one_hot([0,1,2], depth=3)

TensorFlow provides many mathematical functions.

Square

tf.square(x)

Squares every element.

Square Root

tf.sqrt(x)

Requires floating-point numbers.

Natural Logarithm

tf.math.log(x)

Also requires floating-point numbers.

Variable tensors can be updated.

Replace values

x.assign([1,2,3])

Add values

x.assign_add([10,10,10])

TensorFlow tensors can be converted into NumPy arrays.

Example

tensor.numpy()

or

np.array(tensor)

This is useful when using TensorFlow with NumPy libraries.

`@tf.function` converts a normal Python function into an optimized TensorFlow graph.

Example

```
@tf.function
def square(x):
    return x*x
```

Advantages:

- Faster execution
- Optimized computation
- Better performance during model training

TensorFlow can check whether a GPU is available.

Example

```
tf.config.list_physical_devices("GPU")
```

If a GPU is detected, TensorFlow can use it for faster training.

To view GPU details in Google Colab or Linux:

```
!nvidia-smi
```

This shows:

- GPU name
- Memory usage
- Driver version
- CUDA version
- GPU utilization

These are the core TensorFlow fundamentals. Understanding these concepts is essential before moving on to building neural networks, deep learning models, and computer vision applications.